PAGE NO.....
DATE / /

4) Re Bootstrap.

What is Bootstrapping?

⇒ A powerful resampling technique that estimates uncertainty (e.g. standard errors, confidence intervals) without complex math.

- Idea: Treat your sample as a "mini-population" and repeatedly resample from it with replacement.
- No normality assumption needed (unlike traditional CLT-based methods).

How Bootstrapping Works?

1. Take your original sample (e.g. 100 data points).
2. Resample with replacement (same size: 100 points, some repeated).
3. Calculate your statistic (mean, median, model coefficients).
4. Repeat 1,000+ times → Get a distribution of your statistics.
5. Analyse the distribution:
   - a) Standard error = Standard deviation of bootstrap statistics.
   - b) Confidence interval = Middle 95% of bootstrap values.

Key Terms:

- Bootstrap sample: One resampled dataset.
- Resampling: Drawing new samples from your sample (with replacement).

Why Use Bootstrapping?

Traditional Methods	Bootstrap
Requires formulas/math	Pure computation
Assumes normality (for CLT)	Works for any

Limited to simple statistic works for complex metric/models.

Examples Use Cases:

→ Estimating uncertainty for medians, quantiles, or custom metrics.

→ Validating machine learning models (e.g. random forests)

Key Insights

1. **No free lunch:**
   - Bootstrap doesn't fix small samples or biased data.
   - It only estimates how your statistic would vary across samples.

2. **When to use it:**
   - For non-standard statistics (e.g. 90th percentile).
   - When formulas don't exist (e.g., complex ML models).

3. **Computational Cost:**
   - Requires many resample (typically 1,000+).

Common Pitfalls

- Too Few Resamples: <1,000 → Unstable estimates.
- Small Original Sample: Bootstrap can't compensate for $n = 5$.
- Dependent Data: Breaks the independent resamples' assumption (e.g. time series).

DATE / /

Advanced Applications

(1) Bagging (Bootstrap Aggregating):
↳ Train multiple model on bootstrap samples (e.g. random forests).
↳ Average predictions to reduce variance.

(2) Bootstrap Hypothesis Tests:
↳ Simulates null distributions (e.g for p-values).

Final Takeaway

↳ Bootstrapping is like photocopying your data 1,000 times, shuffling each copy, and seeing how much your results wiggle.

python code
```
# Bootstrapping the Mean
import numpy as np
import matplotlib.pyplot as plt

# Original sample (e.g. loan incomes)
(1) { data = np.random.lognormal(10, 1, 1000) # skewed data.

# Bootstrap the median.
# uses append
# we can also use
# List comprehension
n_iterations = 1000
bootstrap_means = []
(2) for _ in range(n_iterations):
        resample = np.random.choice(data, size=len(data),
                                    replace=True)
        bootstrap_means.append(np.median(resample))

# Results
print(f"Original Median: {np.median(data):.2f}")
print(f"Bootstrap SE: {np.std(bootstrap_means):.2f}")
(3) { print(f"95% CI: {np.percentile(bootstrap_means, 2.5):.2f}, {np.percentile
```

bootstrap-median, 97.5):.2f}]"

Plot
plt.hist(bootstrap_medians, bins=30, alpha=0.7)
plt.title("Bootstrap Distribution of Median")
plt.show().

(1) • lognormal(10, 1, 1000).
→ 10 → μ, 1 → σ, 1000 → size
→ If Y ~ Normal(μ=10, σ=1)
then X = exp(Y) follow a log-normal distribution.

→ You're pretending you collected 1,000 people's income.
→ These incomes are skewed (a few super rich),
so using median is smarter than using mean.

(2) Bootstrapping loop.
① for _ in range(n_iterations):
↳ Repeats the process [n_iterations] times (e.g. 1000).

↳ for [_] means "we don't care about the loop index."

② resample = np.random.choice(data, size=len(data), replace=True).
↳ Randomly picks values from [data]

↳ [size=len(data)] → Each sample has the same size as the original dataset (1000 values).

↳ [Replace=True] → sampling with replacement
↳ same value can appear multiple times.

PAGE NO.....
DATE / /

(3) print (f"95% CI: [{np.percentile(bootstrap_medians, 2.5):.2f}, {np.percentile(bootstrap_medians, 97.5):.2f}]")

This finds:
→ The 2.5th percentiles value = lower end of the interval.
→ The 97.5th percentiles value = upper end of the interval.

Together, they form the 95% of CI.

Bootstrapping and permutation test

bootstrapping and permutation tests both use resampling, but they serve different goals and follow different logics.

Feature	Bootstrapping	Permutation Tests
Resampling Method	with replacement	without replacement
what is resampled?	Same sample values	Groups labels between groups.
Goal / Purpose	Estimate variability / CI	Test hypothesis (e.g., is difference real?)
key Output	CI / standard errors	p-value
Assumptions	Few assumptions about data	Also a non-parametric, great for small

DATE / /

(3) print(f"95% CI: [{np.percentile(bootstrap_medians, 2.5):.2f}, {np.percentile(bootstrap_medians, 97.5):.2f}]")

This finds:

→ The 2.5th percentiles value = lower end of the interval.

→ The 97.5th percentiles value = upper end of the interval.

Together, they form the 95% of CI.

Bootstrapping and permutation test both

bootstrapping and permutation tests both use resampling, but they serve different goals and follow different logics.

Feature	Bootstrapping	Permutation Tests.
Resampling Method	with replacement	without replacement.
what is resampled?	Same sample values	Groups labels between groups.
Goal / Purpose	Estimate variability / CI	Test hypothesis (e.g. is difference real?)
key Output	CI / standard errors	p-value
Assumptions	Few assumptions about data	Also a non-parametric, great for small samples.

PAGE NO.....
DATE / /

used when	You want to know how stable your stat is	You want to test if two groups are different
Great Handling for	- Skewed / non-normal data - Building CI - Estimating SE	- Hypothesis testing - checking statistical significance comparing groups without assuming normality.

4(a) Resampling versus Bootstrapping. DATE / /

Q.1 (A) with and without replacement

Analogy

↳ with replacement: After each pick, you put the name back → Same person

↳ without replacement: Names stay out → No repeats.

(B) Why it Matters in Bootstrapping

↳ Creates Variability: some data points appear multiple times, others may be left out.

↳ Mimic the original population: Each bootstrap sample is slightly different, showing how your statistic (eg. mean varies).

Example data: [A, B, C, D]

possible bootstrap sample: [A, B, B, D] (B was picked twice, C was left out).

(C) Used for

with replacement

(1) Preserves data structures: Even with small datasets, you can generate many unique samples.

(2) Estimates uncertainty: shows how your result might change if you collected new data.

without replacement:

is useful for hypothesis tests, not for estimating variability.

(D) Real-word Analogy

- with replacement: Rolling a die multiple times (can get the same number twice).
- without replacement: Drawing cards from a deck (no repeats until reshuffled).

PAGE NO.....
DATE / /

4.8.2 Resampling Methods: Bootstrapping vs Permutation

1. Bootstrapping (Sampling with Replacement).

What it does:
- Treats your sample as a "mini population"
- Randomly draws new samples with replacement (same size as original).
- Repeats 1,000+ times to estimates variability.

When to use it:
- Estimating uncertainty (e.g. confidence intervals for median)
- Small datasets where formula fail.
- Complex statistics (e.g. machine Learning feature importance).

2. Permutation Test (Sampling without Replacement).

What it does:
- Shuffles group labels (e.g. "Treatment" vs "control")
- Re-calculates test statistics for each shuffle.
- Determines if observed difference is statistically significant.

When to use it:
- ✓ Hypothesis testing (e.g. "Do these groups really differ?")
- ✓ Non-normal data where t-test fails.
- ✓ A/B testing scenarios.

PAGE NO.....
DATE / /

Key differences

Feature	Bootstrapping	Permutation tests.
Replacement	with	without
Purpose	Estimate variability	Test hypotheses.
Data Usage	Resamples same data	Shuffle group labels.
Output	Confidence intervals	p-values.

When to choose which.

- Use Bootstrapping with.
  a) You need to quantify uncertainty (e.g. "How precise is this median?").
  b) No formula exists for your statistics.
- Use Permutation Test when:
  a) You want to test if groups are different.
  b) Traditional tests (t-tests) assumption were violated.

Python code